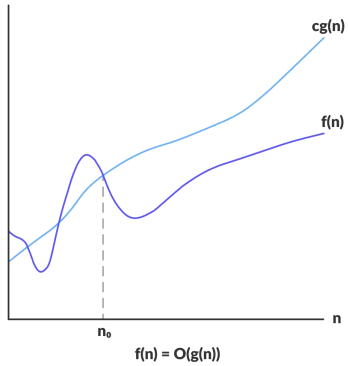
1) A)

Big-O (formal definition).

Let f(n) and g(n) be functions from positive integers to positive reals. We say f(n)=O(g(n)) if there exist constants c>0 and n_0 such that for all $n \geq n_0$,
$f(n) \leq c \cdot g(n)$.



Why important: Big-O gives an asymptotic upper bound that describes how running time or space grows w.r.t. input size. It lets us compare algorithms independent of machine/implementation details and focus on scalability for large n.

1) B)

After k iterations, i = $i=2^{2^k}$. The table below shows values for k=1-5.

k (iteration)	i as expression
1	2^{2^1}
2	2^{2^2}
3	2^{2^3}
4	2^{2^4}
5	2^{2^5}

Termination criterion: loop ends when i >= n.

$$2^{2^k} \geq n$$

$$2^k \geq log_2 n$$

$$k \geq log_2(log_2 n)$$

$$k = O(log_2(log_2 n))$$

Option (iv) is correct.

1) C)

In an unsorted array, we want to determine whether an element is maximum, minimum, or neither maximum nor minimum.

If we take just the first 3 elements, with only 2 comparisons we can identify:

- the maximum element among them,
- the minimum element among them,
- and the third element (which will be neither maximum nor minimum).

Hence, this can be achieved in constant time (independent of n).

Answer: (iv) O(1)

1) D)

$$n^{\log n} \text{ and } (\log n)^n$$

$$\log\left(n^{\log n}\right) \text{ and } \log((\log n)^n) \therefore \text{Apply} \log \text{ on both sides}$$

$$\log n \log n \text{ and } n \log(\log n)$$

$$\text{Put } n = 2^{1024}$$

$$\log 2^{1024} \log 2^{1024} \text{ and } 2^{1024} \log(\log 2^{1024})$$

$$2^{10}.2^{10} \text{ and } 2^{1024}.10$$

$$\therefore 2^{1024}.10 \text{ is greater than } 2^{20}$$

$$\therefore (\log n)^n \text{ grows faster as compared to } n^{\log n}$$

1) E)

Minimum: 1.
(Occurs when the heap has just started filling level)

Maximum: 2^h.
(Occurs when the last level is full)

2)

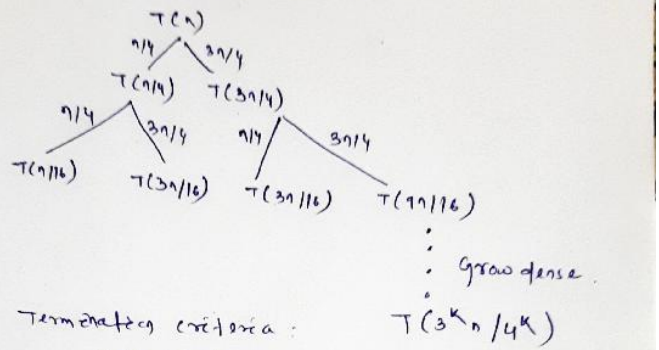
The answer will be $T(n) = O(n \log_{\frac{4}{3}} n)$

If the pivot is the $(n/4)th$ smallest element, partition sizes are roughly $(n/4)$ and $(3n/4)$. The recurrence is:

$$T(n) = T\left(\frac{n}{4}\right) + T\left(\frac{3n}{4}\right) + O(n)$$

This can be solved using recurrence tree method.

(4) $T(n) = T(n/4) + T(3n/4) + n$



Grow dense.

$T(3^k n / 4^k)$

Terminating condition:

$T(3^k n / 4^k) = 1$

$\frac{3^k n}{4^k} = 1$

$\Rightarrow n = \frac{4^k}{3^k}$

$\Rightarrow n = \left(\frac{4}{3}\right)^k$

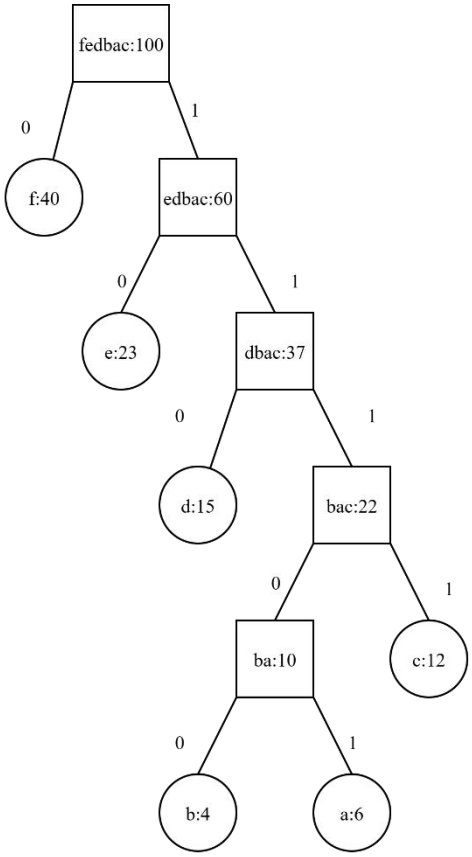
$\Rightarrow k = \log_{4/3} n$

Time complexity = cost per level * number of levels (height of the tree)

$= n * \log_{4/3} n$

$= O(n \log_{4/3} n)$

3)



Character	Frequency	Huffman Code	Code Length
f	40	0	1
e	23	10	2
d	15	110	3
c	12	1111	4
b	4	11100	5
a	6	11101	5

a (6/100) × 5 = 0.30
b (4/100) × 5 = 0.20
c (12/100) × 4 = 0.48
d (15/100) × 3 = 0.45
e (23/100) × 2 = 0.46
f (40/100) × 1 = 0.40
L=2.29 bits/character

For a 2000-character file Huffman-encoded size = 2.29×2000=4580 bits.

Equal-length encoding (3 bits/symbol since $\lceil \log_2 6 \rceil = 3$) = 2000×3=6000 bits.

Bits saved = 6000−4580=1420 bits.

% saving (relative to equal-length) = (1420 / 6000)×100%≈23.67%..

4)

```
 int loss_weight(src, dst, pred[])
{
s = w =1;
while (pred[dst] != src) do
{
w = w x 2;
s = s + w;
dst = pred[dst];
}
return (s);
}
```

Best Case:
When the destination is directly connected to the source → only **1 takeoff**.
Time Complexity: **O(1)**

Worst Case:
When the path traverses through all other vertices → number of takeoffs = **n - 1** (in case of a long chain).
Time Complexity: **O(n)**

OR

Algorithm ComputeTotalWeightLost(pred[], source, destination):

Input: pred[] - Predecessor array

source - Source vertex (New Delhi)

destination - Target capital vertex

Output: total_weight_lost in kilograms

1. Initialize:

current_vertex ← destination

takeoff_count ← 0

2. While current_vertex ≠ source:

current_vertex ← pred[current_vertex]

takeoff_count ← takeoff_count + 1

3. total_weight_lost ← (2^takeoff_count) - 1

4. Return total_weight_lost

Explanation

Start at the destination vertex.

Move backwards using pred[] until we reach the source (New Delhi).

Count the number of takeoffs (edges) traversed → takeoff_count.

Apply the geometric series formula for weight loss:
Weight Lost = 2^k - 1, where k = takeoff_count.

Suppose:
- source = 0 (New Delhi)
- destination = 5
- pred[5] = 3, pred[3] = 1, pred[1] = 0

Then:

Number of takeoffs = 3

Total weight lost = $2^3 - 1 = 7$ kg

5) A)

$$T(n) = a \cdot T\left(\frac{n}{b}\right) + f(n)$$

where:

- $a = 3$
- $b = 3$
- $f(n) = n^2$

Compute $n^{\log_b a}$:

$$\log_b a = \log_3 3 = 1$$

So, $n^{\log_b a} = n^1 = n$.

Now, compare $f(n)$ and $n^{\log_b a}$:

- $f(n) = n^2$ grows **faster** than n.
- Therefore, this is **Case 3** of the Master Theorem, where $f(n) = \Omega(n^{\log_b a + \epsilon})$ for some $\epsilon > 0$, and the regularity condition $a \cdot f(n/b) \leq c \cdot f(n)$ for some constant $c < 1$ holds.

Check the regularity condition:

$$a \cdot f\left(\frac{n}{b}\right) = 3 \cdot \left(\frac{n}{3}\right)^2 = 3 \cdot \frac{n^2}{9} = \frac{n^2}{3}$$

And $f(n) = n^2$.

So:

$$a \cdot f\left(\frac{n}{b}\right) = \frac{n^2}{3} \leq c \cdot n^2$$

This holds for any constant $c \geq \frac{1}{3}$ and $c < 1$.

Therefore, the regularity condition holds.

By **Master Theorem Case 3**:

$$T(n) = \Theta(f(n)) = \Theta(n^2)$$

B)

job	a	b	c	d	e	f	g	h	i
profit	15	20	30	18	18	10	23	16	25
deadline	7	2	5	3	4	5	2	7	3

Slot	**Job**	**Profit**
1	b	20
2	g	23
3	i	25
4	e	18
5	c	30
6	a	15
7	h	16

Optimal Greedy Schedule Profit =20+23+25+18+30+15+16=147.